



*Advanced AI-based dynamic Integration
of heterogeneous data for semantic
clusters formalisation in human-centric
intelligent enterprise systems*

Pokemon Ontology NLP Processing

Mario Lezoche

Thomas Pallet

Monday 26th January 2026

Table of content

01

**Text
preprocessing**

02

**Entity
recognition &
linking**

03

**Relation
extraction**

04

**Instance
creation & RDF
export**

05

**Semantic
clustering &
validation**

06

Conclusion

Table of content

01

**Text
preprocessing**

02

**Entity
recognition &
linking**

03

**Relation
extraction**

04

**Instance
creation & RDF
export**

05

**Semantic
clustering &
validation**

06

Conclusion

Generated text

A short story in the Kanto region follows.

Ash is a trainer who travels with a Pikachu. He meets a Bulbasaur in the forest and a Charmander near a rocky path.

Later, a Squirtle appears by the river and shows a Water Gun move. Ash watches Pikachu use Thunder Shock against a wild Pidgey.

The group rests near a lake where a Charizard flies above and a Venusaur moves through tall grass.

Ash notes the types he sees: Electric for Pikachu, Grass and Poison for Bulbasaur, Fire for Charmander, Water for Squirtle.

He writes in his journal that Flamethrower is a Fire move and Vine Whip is a Grass move.

Step 1: cleaning

Remove noise characters and normalize whitespace

Raw → Cleaned

Ex:

“Later, a Squirtle appears by the river and shows a Water Gun move”

→

“Later a Squirtle appears by the river and shows a Water Gun move”

Step 2: segmentation

Segmentation of sentence

Text → List of sentences

Ex:

sentence1 : “A short story in the Kanto region follows.”

sentence2 : “Ash is a trainer who travels with a Pikachu.”

...

Step 3: tokenization

Tokenization of each words inside a sentence

Sentence → List of tokens

Ex:

sentence : “Ash is a trainer who travels with a Pikachu.”

→

tokens : [Ash, is, a, trainer, who, travels, with, a, Pikachu]

Step 4: lemmatization

Normalization of tokens to their base form (moves -> move)

List of tokens → List of lemmas

Ex:

tokens : [Ash, is, a, trainer, who, travels, with, a, Pikachu]

→

lemmas : [ash, be, a, trainer, who, travel, with, a, pikachu]

Step 5: stopwords

Removal of noise

List of lemmas → filtered list of lemmas

Ex:

lemmas : [ash, be, a, trainer, who, travel, with, a, pikachu]

→ stopwords : [is, a, who, with]

filtered lemmas : [ash, trainer, travel, pikachu]

Text after phase 1

A short story in the Kanto region follows.

Ash is a trainer who travels with a Pikachu.

He meets a Bulbasaur in the forest and a Charmander near a rocky path.

Later, a Squirtle appears by the river and shows a Water Gun move.

Ash watches Pikachu use Thunder Shock against a wild Pidgey.

The group rests near a lake where a Charizard flies above and a Venusaur moves through tall grass.

Ash notes the types he sees: Electric for Pikachu, Grass and Poison for Bulbasaur, Fire for Charmander, Water for Squirtle.

He writes in his journal that Flamethrower is a Fire move and Vine Whip is a Grass move.

short story kanto region follow

ash be trainer travel pikachu

meet bulbasaur forest charmander near rocky path

later squirtle appear river show water gun move

ash watch pikachu use thunder shock against wild pidgey

group rest near lake where charizard fly above venusaur
move through tall grass

ash note type see electric pikachu grass poison bulbasaur
fire charmander water squirtle

write journal flamethrower be fire move vine whip be grass
move

Table of content

01

**Text
preprocessing**

02

**Entity
recognition &
linking**

03

**Relation
extraction**

04

**Instance
creation & RDF
export**

05

**Semantic
clustering &
validation**

06

Conclusion

Step 1: build ontology lexicon

Extract labels/names from OWL

OWL → Dict : List IRI

Ex:

```
{  
...  
“pikachu” → http://example.org/pokemon#species_0025,  
...  
  
“thunder shock” → http://example.org/pokemon#ThunderShock,  
...  
}
```

Step 2: lexicon detection

Search in text phrases that match lexicon (longest match first)

Raw text + Lexicon → List of detected mentions (with position)
raw text to make sure that we know where it comes from

Ex:

<pre>{ "text": "Pikachu", "start": 79, "end": 86, "label": "pikachu", "iris": ["http://example.org/pokemon#ind_00000001", "http://example.org/pokemon#species_0025"] },</pre>	<pre>{ "text": "Pikachu", "start": 138, "end": 245, "label": "pikachu", "iris": ["http://example.org/pokemon#ind_00000001", "http://example.org/pokemon#species_0025"] },</pre>	<pre>{ "text": "Pikachu", "start": 428, "end": 435, "label": "pikachu", "iris": ["http://example.org/pokemon#ind_00000001", "http://example.org/pokemon#species_0025"] },</pre>
---	---	---

Step 3: entity linking

Attach each mention to one or more ontology IRIs.

Mentions + **Lexicon** → **Linked mentions (IRIs)**

Ex:

{

...

“pikachu” → pokemon:species_0025,

...

“thunder shock” → pokemon:ThunderShock,

...

}

Step 4: sentence grouping

Group mentions by sentence to keep local context for later relation extraction

Linked mentions + sentence → **mentions per sentence**

Ex:

```
{
  "sentence": "ash watch pikachu use thunder shock against wild pidgey",
  "mentions": [
    { "text": "Pikachu", "iris": ["http://example.org/pokemon#species_0025"], ... },
    { "text": "Thunder Shock", "iris": ["http://example.org/pokemon#ThunderShock"], ... }
  ]
}
```

Table of content

01

**Text
preprocessing**

02

**Entity
recognition &
linking**

03

**Relation
extraction**

04

**Instance
creation & RDF
export**

05

**Semantic
clustering &
validation**

06

Conclusion

Step 1: identify entity types

Mentions + Ontology → IRIs classified as PokemonSpecies/Move/type

Ex:

```
{  
...  
“pokemon:species_0025” → PokemonSpecies,  
...  
“pokemon:ThunderShock” → Move,  
...  
}
```

Step 2: applied relation rules

Use of trigger words (ex: use/uses/show/shows → canLearnMove).

If ambiguous, then check domain/range, can also check the nearest pairs etc...

Lots of improvement possible

Sentence + classified entities → triples

Ex:

Pikachu use Thunder Shock

→

pokemon:species_0025 pokemon:canLearnMove pokemon:ThunderShock

Table of content

01

**Text
preprocessing**

02

**Entity
recognition &
linking**

03

**Relation
extraction**

04

**Instance
creation & RDF
export**

05

**Semantic
clustering &
validation**

06

Conclusion

Ontology uses

**Creation of RDF triples when querying the graph.
And store that in memory to have a big set of triples**

Ex:

`pokemon:ThunderShock rdf:type pokemon:Move`

`pokemon:species_0025 rdfs:subClassOf pokemon:PokemonSpecies`

Instance creation

Needs: Ontology +

- **entities detected (step 2)**
- **relations between entities (step 3)**

→ **Creates a new instance per pokemon species mention**

→ **Adds rdf:type and pokemon:name attributes**

→ **Attaches extracted relations to those instances**

Ex:

```
pokemon:ind_auto_0006 a pokemon:species_0025 ;
```

```
    pokemon:canLearnMove pokemon:ThunderShock ;
```

```
    pokemon:hasType pokemon:Electric,
```

```
    pokemon:Fire,
```

```
    pokemon:Grass,
```

```
    pokemon:Poison,
```

```
    pokemon:Water ;
```

```
    pokemon:name "Pikachu" .
```

Table of content

01

**Text
preprocessing**

02

**Entity
recognition &
linking**

03

**Relation
extraction**

04

**Instance
creation & RDF
export**

05

**Semantic
clustering &
validation**

06

Conclusion

Limitations

Possible to check names for instances for example in order to not merge them if already exists (because names generated automatically)

Trigger-based bias: ambiguity in text (travels with here = owns but not always)

Need to check entity constraints

Improvements

Can build confidence score :

- Build a heuristic score from: label match, domain/range compatibility, trigger word presence, and proximity in the sentence.
- Use thresholds to auto-accept high scores and review uncertain ones.

Stronger tools (for automated extraction):

- **spaCy dependency parsing**: extract subject–verb–object relations more precisely than co-occurrence rules.
- **NER + entity linking models**: robust detection and linking beyond exact string match.
- **Supervised relation extraction (BERT-like)**: classify relations between two entities given context.
- **OpenIE**: extract triples without predefined predicates, then map to ontology.
- **Knowledge-graph frameworks**: Snorkel (weak supervision), DeepKE, Stanford OpenIE for full KG pipelines.

Thank you for your attention !

Questions ?